

```
Terminal
File Edit View Tools Extension Help Run
def birthday():
    return render_template('birthday.html')
@app.route('/about')
def about():
    return render_template('about.html')
@app.route('/check_birthday', methods=['POST'])
def check_birthday():
    user_month = request.form['month']
    user_day = request.form['day']
    dt_obj = datetime.now()
    today_month = dt_obj.strftime('%B')
    today_day = dt_obj.strftime('%d')
    check_month = today_month == user_month
    check_day = today_day == user_day
    if check_month and check_day:
        msg = 'Happy birthday!'
        txt = 'Today is your birthday. Spend it doing something you enjoy with those you enjoy being around.'
    else:
        msg = 'It is not your birthday'
        txt = 'Sadly, today is not your birthday. Try again later.'
    return render_template('check.html', values(msg, txt))
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=3800, debug=True)
```

```
Slide
Collapse
```

Deploying with Heroku

What is Heroku?

We have our Flask app and Dockerfile, which means we are ready to build and deploy our Docker image. We are going to use Heroku for this. Heroku is one of the first cloud computing systems. They provide a platform as a service (PaaS) to their customers. This means Heroku builds out and maintains an infrastructure that customers can use to host their own applications. Our Flask app needs to run on a web server to be accessible to the wider internet. Heroku provides all of the infrastructure needed to host a website. They provide the servers, they offer a set of installable software, they provide software updates to ensure a secure system, they generate a URL for our Flask app, etc.

The first thing you need to do is go to Heroku and create a free account. You may be asked to enable two-factor authentication for your account. So this process could be a bit longer than you would expect.

Once we have our Heroku account, we need to download and install the Heroku command line interface (CLI). Codo runs on the Ubuntu operating system, so run the following script in the terminal. You should see a success message after installation.

```
curl https://cli-assets.heroku.com/install-ubuntu.sh | sh
```

We need to login to our Heroku account via the CLI before we can go any further. Heroku will ask for your username and password. However, if your account is set up with multifactor authentication, they will not let you authenticate with this information. Instead, you have to use an API key in place of your password. Go to the tab where you are logged in on Heroku. Click your avatar in the top-right corner. Select account settings. Scroll down until you see the section for API Key. Copy the API key. You do not need to click reveal first. You can copy the obscured API key.

API key

Reveal

Regenerate API key

Once we have the key, we can start the login process. In the terminal, enter the following command.

```
heroku login -i
```

You will be prompted for your email and password for Heroku. **Remember**, use the API key in place of your password. The process will look something like the example below.

```
heroku: Enter your login credentials
Email: your-email@provider.com
Password: *****
Logged in as your-email@provider.com
```

Next, we need to create our Heroku application. The name used for the application will also be used in the URL for our Flask app. So the application name needs to be unique to you. The command below creates a Heroku application and sets the buildpack to Python. **Important**, replace <year> with your initials or some other unique identifier.

▼ Unique names
If the name you chose for your app is not unique, you will see a message that this name is already taken. You will have to run heroku create again with a different name. You may have to do this a few times until you find a unique name.

```
heroku create check-birthday-<your initials> --buildpack heroku/python
```

Upon successful application creation, you will see a message like the one below. You get a URL for the Heroku app where we can see our Flask app running on the internet. The other URL is for the git repository where our code lives. Take note of the first URL.

```
Creating check-birthday... done
https://check-birthday.herokuapp.com/ | https://git.heroku.com/check-birthday.git
```

By default, Heroku runs Ubuntu for its applications. We want to run a Docker container instead. Set the stack to container and use the -a flag to name the application to which this change applies. **Important**, replace <heroku app name> with the name of the Heroku app you chose above.

```
heroku stack:set container -a <heroku app name>
```

Heroku YAML File and Flask App

Now that we have setup and configured our Heroku app, we need Heroku to build our Docker image and deploy it to a container. This is done with a YAML file called heroku.yml. Enter the following command to create the the YAML file. Then open it with the link below.

```
touch heroku.yml
```

Open heroku.yml.

This file serves as a set of instructions for building and deploying our Flask app. The first section is for building. Specify that we are building a Docker image and then use our Dockerfile for the build process. Normally we would follow the build section with a run section. However, by omitting the run section Heroku will default to our `entrypoint` command in our Dockerfile instead. That's all we need for the YAML file.

```
build:
  docker:
    web: Dockerfile
```

The last few changes come to our `main.py` app. Previously, our Flask app was setup to run in debug mode on a specific port (3000 in our case). We need to turn off debug mode and let Heroku specify what port it wants to use. Open the Python file with the link below. Then, import the `os` module.

Open main.py

```
import os
from datetime import datetime
from flask import Flask, render_template, request
```

In the conditional at the bottom, we want to create the variable `port`. This will get the `PORT` environment variable from Heroku. Then use the `port` variable for setting the port and change `debug` to `False`.

```
if __name__ == '__main__':
    port = os.environ.get('PORT', 5000)
    app.run(host='0.0.0.0', port=port, debug=False)
```

All of our files are finally done. The next step is to put all of this on Heroku, build our Docker image, and deploy the Docker container on the internet. Remember, Heroku provides each application with a git repository. As such, we are going to use git commands to finalize this process. Start by initializing the directory for git.

```
git init
```

Next, we want to add all of our files so they will be pushed to Heroku.

```
git add .
```

We need to commit our changes. Use the `-m` flag to write a message about the commit. Since this is our first commit, "initial commit" is a sufficient message. If you make any future changes, use descriptive commit messages to help you remember the changes you made.

```
git commit -m "Initial commit"
```

Finally, push our changes to the git repository. This will kick of a series of events that build our Docker image, create a container, and ultimately lead to our Flask app running on the internet.

```
git push heroku master
```

This process can take a bit of time to complete. Once done, open a new tab and go to the Heroku URL to see the Flask app running.

▼ What if I forgot my URL?
Go to your Heroku dashboard, which has a list of all your applications. Click on your Flask app. Toward the top-right, click the button that says 'open app'. This will open your Flask app in a new tab.

Reading Question

Select all of the reasons why you would use a platform as a service (PaaS) like Heroku. **Hint**, there is more than one correct answer.

- ☐ Security - you have control over the infrastructure so you can make sure every piece of software is up to date.
- ☐ Independence - you are not reliant on others for mission critical software and services.
- ☐ Speed up development - the platform provides automated tools like building the Dockerfile.
- ☐ Cost - you do not need to purchase and maintain a technology stack (servers, software, etc.)

The correct answers are:

- Cost - you do not need to purchase and maintain a technology stack (servers, software, etc.)
- Speed up development - the platform provides automated tools like building the Dockerfile

PaaS, like Heroku, provide a platform which you can use without having to purchase and maintain yourself. This can lead to a reduction in cost. In addition, you can speed up development time using the platform. For example, we did not have to tell Heroku to explicitly build the Docker image and then spin up the container. All of this happened automatically when we pushed our code to the git repository.

However, you become reliant on the platform provider. They are the ones who maintain the platform, so it is up to them to ensure the platform is secure. There is an inevitable loss of independence when using a PaaS.

ANSWER

Next